

Database Management System

Key topics

- Big Data
- Storage System
- Parallel Processing
- Data replication and consistency
- Streaming data
- Graph database
- NoSQL system

Big Data

refers to a extremely large data-set which cannot be effectively managed

Characteristic →

volume → Petabytes or exabytes

velocity → real time data stream

variety → structured, semi-structured
and unstructured

The two main system for handling big data are →

* Transaction Processing System

* Query Processing System

Transaction Processing System

1. Balancing Efficiency and Integrity:

- TPS are designed to **process a high volume of transactions efficiently** while maintaining **data integrity (ACID properties)**.
- However, maintaining integrity (like consistency and isolation) often **slows down the system**, especially in a distributed environment.

2. Scalability vs. ACID Compliance:

- **Scalability** demands that the system can handle more transactions as load increases.
- **ACID compliance** requires careful synchronization and data locking.
- These two requirements conflict because ensuring strict consistency in a distributed environment can cause performance bottlenecks.

3. Reliability vs. Cost:

- Ensuring **reliability and fault tolerance** requires robust infrastructure, which can be **expensive and complex to maintain**.
- The system's ability to **quickly recover from failures** makes it reliable, but achieving that often requires **costly redundancy and failover mechanisms**.

4. Concurrency vs. Contention:

- To process many transactions simultaneously (high concurrency), the system must **lock resources carefully** to maintain data integrity.
- This locking can cause **contention and slow performance** when many transactions compete for the same data.

Query Processing System :-

This system is designed to efficiently execute queries on large dataset

- * Focuses on data retrieval and analysis

Pros :-

- * High Performance in data retrieval
- * Flexible
- * Efficient data aggregation
- * Scalability
- * Cost optimization

Cons :-

- * Complex Optimization
- * High resource consumption
- * Limited acid compliance
- * Data redundancy and Inconsistency
- * Latency in real time processing

Big Data Storage System

The storage system is designed to store, manage and retrieve vast amount of data efficiently

To store big data

- Files are distributed across various nodes
- (Sharding) splitting large datasets into smaller, manageable parts
- Stores the data in key value pairs
- spreading the data and computation across multiple server

Distributed File System

- * Stores data across large collection of machine while providing single file system
- * Provides redundant storage by utilizing cheap and spread across computers

Example → 1) Google File System
2) Hadoop File System

Hadoop File System Architecture

Key Components →

- * Name Node

- * Data node

Additional components

- Secondary Name Node

- Check point node

- Back up node

1. NameNode (Master Node):

- **Role:** Manages the metadata and file system namespace.
- **Responsibilities:**
 - Maintains the **directory structure** and **file-to-block mapping**.
 - Keeps track of the **locations of data blocks**.
 - Manages the **access control** and **permissions**.
 - Monitors the **health of DataNodes**.
- **Storage:** Stores metadata in two files:
 - **FsImage:** A snapshot of the file system metadata.
 - **EditLogs:** A record of every file system change.
- **Single Point of Failure (SPOF):** If the NameNode fails, HDFS becomes inaccessible.

2. DataNode (Slave Node):

- **Role:** Stores the actual data blocks.
- **Responsibilities:**
 - Stores data as **blocks** (default size: 128 MB or 64 MB).
 - Regularly **sends heartbeats** to the NameNode to indicate health.
 - Handles **read and write requests** from clients.
 - Performs **block replication** as instructed by the NameNode.
- **Storage:** Each block is stored as a **separate file** on the local file system.

3. Secondary NameNode:

- **Role:** Acts as a **checkpoint node**, not a backup.
- **Responsibilities:**
 - Periodically merges the **FsImage** and **EditLogs**.
 - Reduces the size of **EditLogs**, preventing them from becoming too large.
- **Note:** Does not act as a replacement if the primary NameNode fails.

📁 4. Checkpoint Node:

- Combines FsImage and EditLogs to create a new checkpoint.
- Reduces the size of EditLogs and keeps the metadata consistent.

🛡️ 5. Backup Node:

- Maintains an up-to-date copy of the FsImage in memory.
- Receives real-time updates from the NameNode.
- Can be used to restart the NameNode without data loss.

* **EditLogs** → A file that records every changes made to the HDFS file system

* **Fs Image** → A snapshot of the entire file system meta data

📁 HDFS Data Flow:

1. File Write:

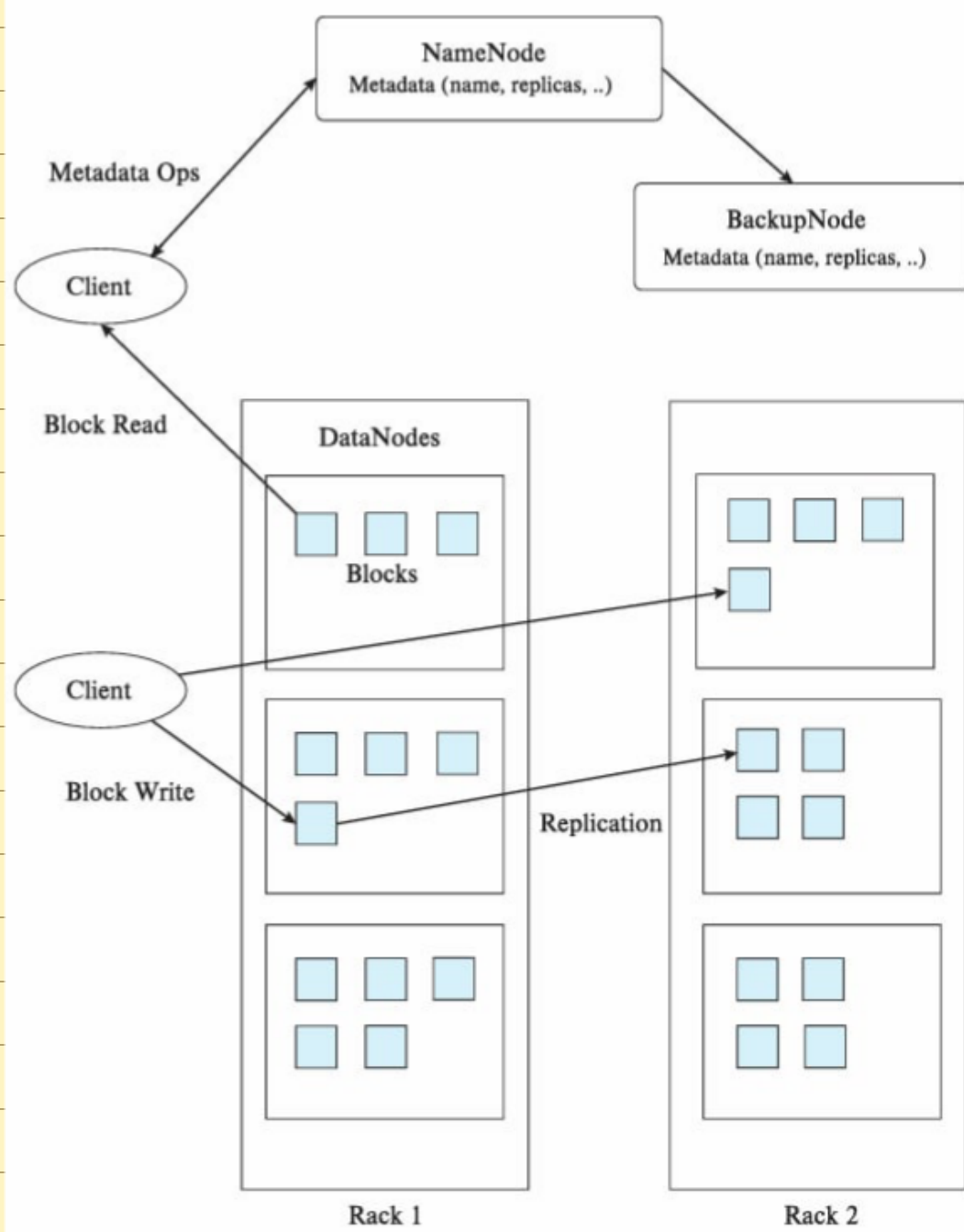
- The client sends a **write request** to the NameNode.
- The NameNode allocates a **block ID** and determines the **DataNodes** for replication.
- The client writes data to the first DataNode, which **pipeline**s it to **other replicas**.

2. File Read:

- The client sends a **read request** to the NameNode.
- The NameNode responds with the **list of DataNodes** containing the blocks.
- The client directly reads the data from the nearest DataNode.

3. Data Replication:

- Default replication factor: 3.
- Ensures **data availability** and **fault tolerance**.
- If a DataNode fails, the NameNode initiates **replication** from **surviving nodes**.



Sharding { Type of horizontal scaling }

a method of distributing data across multiple database instances

- * Each small, manageable database instances are called shards

how sharding works →

- * Data is partitioned based on attributes

Known as partition key

- * The keys also determine how data is distributed among shards

- * The application knows which shard contains which data and run queries / updates data like wise

Pros →

- * Scalable
- * Easy to implement

Cons →

- * Lack of transparency

→ the application must handle query routing and managing queries that span multiple shard is complex

* Overloading Issues

→ when a shard is overloaded, redistributing data brings hardship causing data migration and downtime

* High risk of failure

Here, more the shards more the chance of failure so to ensure availability of data replicas are maintained

* Sharding system doesn't support join operations, integrity constraints etc across partition

To increase scalability 4 approach was taken

→ adding memcache or caching mechanism to application servers

→ Sharding

→ using existing parallel databases designed for DSS not OLTP

→ Using massively parallel key value data store

Key value Storage System / NoSQL

Key value storage system completely follows sharding but there are some constraint in this,

- * stores large number of small (KB-MB) records

- * Records are replicated among multiple machine to ensure data consistency.

What it stores →

- 1) uninterpreted bytes
 - 2) Wide table
 - 3) JSON
- } with associated key

Operations → put (key, value), get (key), delete (key)

This system also support range queries in key value

The MapReduce Paradigm

A model for performing parallel and distributed computing efficiently

Example → Google Map Reduced
→ Hadoop

How it works →

Let's pass a string
 $S = \text{"Say one one time"}$

Map will take string key and records

$K = 0$ and $V = S$ $\text{map}(K, V)$

then the output of the map will be

$\text{list} = \{ \underbrace{\{\text{Say}, 1\}}_{\text{key}}, \underbrace{\{\text{one}, 1\}}_{\text{value}}, \{\text{one}, 1\}, \{\text{time}, 1\} \}$

Then this list goes through shuffle and sort phase and produces grouping by key

{ Say, [1]}

{ One, [1,1]}

{ time, [1]}

and sorts it self

{ One, [1,1]}

{ Say, [1]}

{ time, [1]}

Then it goes through reduce

reduce (k, list(v_i))

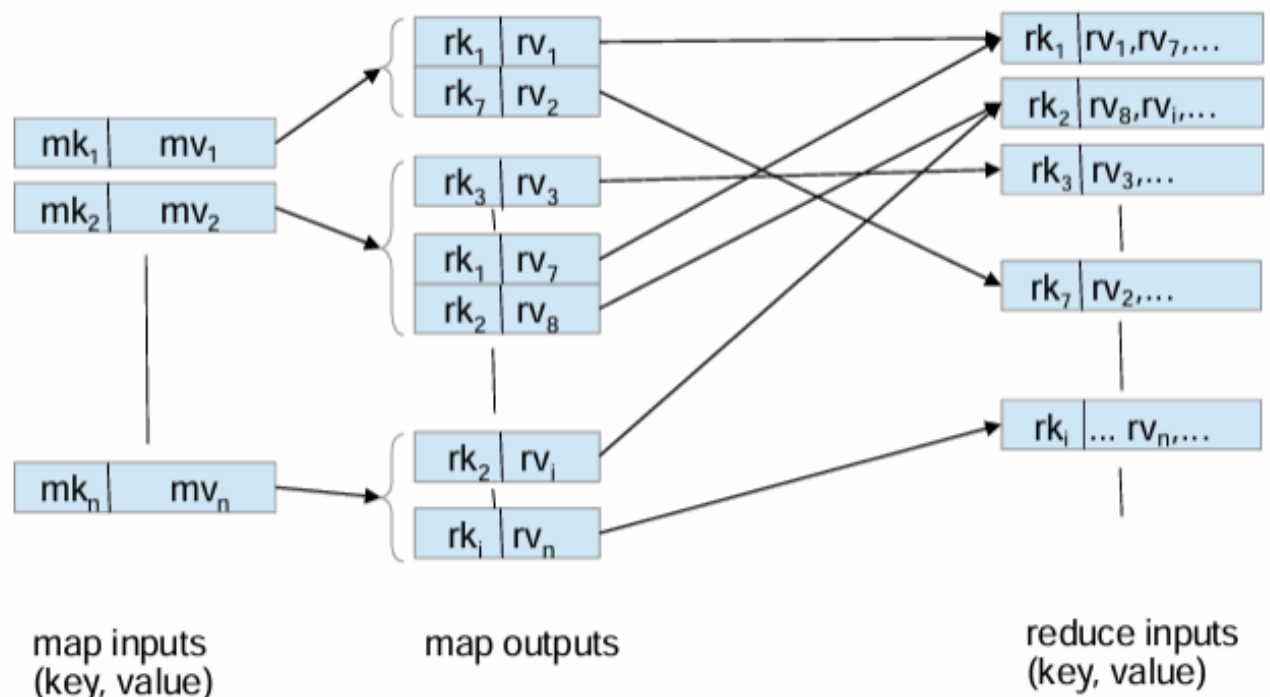
and generates

{ One, 2 }

{ say, 1 }

{ time, 1 }

Flow of keys and values in a map reduce task

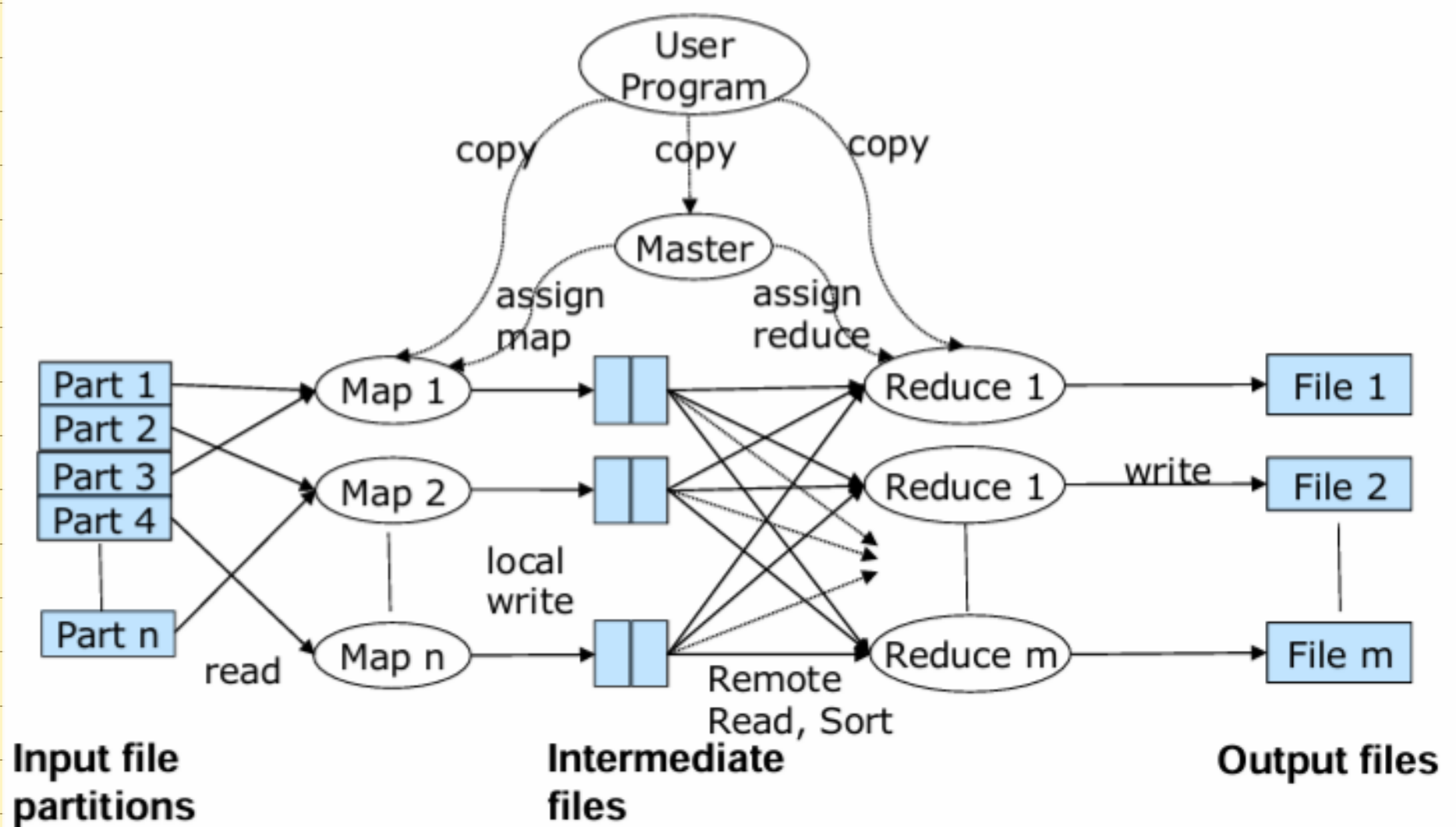


How user interacts with it?

- User submits a mapReduce job
- Master node co-ordinate the whole task assigns to mappers and reducers and monitors them
- Worker nodes are mappers and reducers
- Storage system has input files, intermediate files and output files

Execution

- * Input data is split into fixed size blocks
- * Then master node assigns each idle workers task
- * Workers do their job
- * Reducers stores final output in HDFS



Hadoop MapReduce

Map Reduce implementation in java

- * this uses HDFS (Hadoop file system)
- * takes input in Text / CSV
- * Supports key value stores such as Hbase, Cassandra, MongoDB etc

Read Till slide 32

Streaming Data

- * a continuous, real time data flow

Queries on stream →

- * windowing { break streams into windows to run queries, this windows are made based on time stamp or tuples }

- * Continuous Queries { outputs results based on the existing data that has streamed, the output is adjusted gradually }

Stream allows algebraic operations and pattern matching

Stream uses lambda architecture

The **Lambda architecture** addresses this challenge by splitting data processing into **two parallel paths**:

Path	Layer	Technology Examples	Pros & Cons
Speed Layer	Stream Processing	Apache Storm, Flink, Kafka	Low latency , but may miss historical data.
Batch Layer	Database Storage	Hadoop, HDFS, Cassandra	Accurate (full dataset), but high latency.

3. How Lambda Architecture Works

1. Data Ingestion:

- A single data stream (e.g., Twitter tweets) is **split into two copies**.
 - **Copy 1**: Sent to the **speed layer** for real-time processing.
 - **Copy 2**: Stored in the **batch layer** for comprehensive analysis.

2. Speed Layer (Real-Time):

- Processes data **in-memory** with minimal delay.
- Outputs approximate results (e.g., trending hashtags updated every second).
- *Example*: Apache Storm counts tweets per hashtag in real-time.

3. Batch Layer (Historical):

- Data is persisted in a database (e.g., HDFS).
- Runs periodic batch jobs (e.g., nightly) to compute **accurate results** (e.g., daily hashtag counts).
- *Example*: Hadoop aggregates all tweets over 24 hours.

4. Serving Layer:

- Combines results from both layers for queries.
- *Example*: A dashboard shows both real-time trends and historical comparisons.

Read From Slide 37